P1072R1
Optimized Initialization for basic_string and vector

Published Proposal, 2018-10-07

This version:
http://wg21.link/P1072R1

Authors:
Chris Kennelly (Google)
Mark Zeren (VMware)

Audience:
LEWG, LWG, SG16

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Allow access to uninitialized or default initialized elements when working with basic_string and vector.

Table of Contents

§ 1. Motivation

We teach that `vector` is a "sequence of contiguous objects". With experience, users learn that `vector` is also a memory allocator of sorts -- it holds uninitialized capacity where new objects can be allocated inexpensively. `vector::reserve` manages this capacity directly. `basic_string`

provides similar allocation management but adds a null termination invariant and, frequently, a small buffer optimization (SBO).

Both `vector` and `basic_string` provide an invariant that the objects they control are always value, direct, move, or copy initialized. It turns out that there are other ways that we might want to create objects.

Performance sensitive code is impacted by the cost of initializing and manipulating strings and vectors: When streaming data into a `basic_string` or a `vector`, a programmer is forced with an unhappy choice:

- Pay for extra initialization (`resize` then copy directly in)
- Pay for extra copies (populate a temporary buffer, copy it to the final destination)
- Pay for extra "bookkeeping" (`reserve` followed by small appends)

C++'s hallmark is to write efficient code by construction and this proposal seeks to enable that.

Sometimes, it is useful to manipulate strings without paying for the bookkeeping overhead of null termination or SBO. This paper proposes a mechanism to transfer ownership of a `basic_string`'s memory "allocation" (if it has one) to a "compatible" `vector`. After manipulation, the allocation can be transferred back to the string.

We present three options for LEWG's consideration:

- A full-fledged container type, `storage_buffer` (§3.1 storage_buffer as a container), for manipulating uninitialized data
- A transfer-orientated node-type, `storage_node` (§3.2 storage_node as a transfer vehicle), for moving buffers between `basic_string` and `vector`, relying on either
  - Additions to `vector` by [P1010R1] to allow for access to uninitialized data, then marking it as committed (`insert_from_capacity`).
  - The user to allocate a buffer, populate it, then pass it to the node-type for transfer into this ecosystem.

§ 1.1. Stamping a Pattern

Consider writing a pattern several times into a string:

```
std::string GeneratePattern(const std::string& pattern, size_t count) {
   std::string ret;

   ret.reserve(pattern.size() * count);
   for (size_t i = 0; i < count; i++) {
     // SUB-OPTIMAL:
     // * Writes 'count' nulls
     // * Updates size and checks for potential resize 'count' times
     ret.append(pattern);
   }

   return ret;
}
```

Alternatively, we could adjust the output string's size to its final size, avoiding the bookkeeping in `append` at the cost of extra initialization:

```
std::string GeneratePattern(const std::string& pattern, size_t count) {
   std::string ret;

   const auto step = pattern.size();
   // SUB-OPTIMAL: We memset step*count bytes only to overwrite them.
   ret.resize(step * count);
   for (size_t i = 0; i < count; i++) {
     // GOOD: No bookkeeping
     memcpy(ret.data() + i * step, pattern.data(), step);
   }

   return ret;
}
```

We propose three avenues to avoid this tradeoff. The first possibility is `storage_buffer`, a full-fledged container providing default-initialized elements:

```
std::string GeneratePattern(const std::string& pattern, size_t count) {
   std::storage_buffer<char> tmp;

   const auto step = pattern.size();
   // GOOD:  No initialization
   tmp.prepare(step * count + 1);
   for (size_t i = 0; i < count; i++) {
     // GOOD: No bookkeeping
     memcpy(tmp.data() + i * step, pattern.data(), step);
   }

   tmp.commit(step * count);
   return std::string(std::move(tmp));
}
```

Distinctly, `storage_buffer` is move-only, avoiding (depending on `T`) potential UB from copying uninitialized elements.

The second possibility is `storage_node` as a node-like mechanism (similar to the now existent API for associative containers added in [P0083R3]) for transferring buffers, coupled with new APIs for `std::vector` from [P1010R1] but also merged in this paper).

```
std::string GeneratePattern(const std::string& pattern, size_t count) {
   std::vector<char> tmp;

   const auto step = pattern.size();
   // GOOD:  No initialization
   tmp.reserve(step * count + 1);
   for (size_t i = 0; i < count; i++) {
     // GOOD: No bookkeeping
     memcpy(tmp.uninitialized_data() + i * step, pattern.data(), step);
   }

   tmp.insert_from_capacity(step * count);
   return std::string(tmp.extract()); // Transfer via storage_node.
}
```

§ 1.2. Interacting with C

Consider wrapping a C API while working in terms of C++'s `basic_string` vocabulary:

```
std::string CompressWrapper(std::string_view input) {
    std::string compressed;
    // Compute upper limit of compressed input.
    size_t size = compressBound(input.size());

    // SUB-OPTIMAL:  Extra initialization
    compressed.resize(size);
    int ret = compress(compressed.begin(), &size, input.data(), input.size(
    if (ret != OK) {
      throw ...some error...
    }

    // Shrink compress to its true size.
    compress.resize(size);
    return compressed;
}
```

With the proposed `storage_buffer`:

```
std::string CompressWrapper(std::string_view input) {
    std::storage_buffer<char> compressed;
    // Compute upper limit of compressed input.
    size_t size = compressBound(input.size());

    // GOOD:  No initialization
    compressed.prepare(size + 1);
    int ret = compress(compressed.begin(), &size, input.data(), input.size(
    if (ret != OK) {
      throw ...some error...
    }

    // Shrink compress to its true size.
    compress.commit(size);
    return std::string(std::move(compressed));
}
```

With `vector` and `storage_node`:

```
std::string CompressWrapper(std::string_view input) {
    std::vector<char> compressed;
    // Compute upper limit of compressed input.
    size_t size = compressBound(input.size());

    // GOOD:  No initialization
    compressed.reserve(size);
    int ret = compress(compressed.begin(), &size, input.data(), input.size(
    if (ret != OK) {
      throw ...some error...
    }

    // Shrink compress to its true size.
    compress.insert_from_capacity(size);
    return std::string(std::move(compressed)); // Transfer via storage_node
}
```

§ 2. Related Work

§ 2.1. Google

Google has a local extension to `basic_string` called `resize_uninitialized` and is wrapped as `STLStringResizeUninitialized`.

- [Abseil] uses this to avoid bookkeeping overheads in `StrAppend` and `StrCat`.
- [Snappy]
    - In decompression, the final size of the output buffer is known before the contents are ready.
    - During compression, an upperbound on the final compressed size is known, allowing data to be efficiently added to the output buffer (eliding `append`'s checks) and the string to be shrunk to its final, correct size.
- [Protobuf] avoids extraneous copies or initialization when the size is known before data is available (especially during parsing or serialization).

§ 2.2. MongoDB

MongoDB has a string builder that could have been implemented in terms of `basic_string` as a return value. However, as explained by Mathias Stearn, zero initialization was measured and was too costly. Instead a custom string builder type is used:

E.g.: https://github.com/mongodb/mongo/blob/master/src/mongo/db/fts/unicode/string.h

```
/**
 * Strips diacritics and case-folds the utf8 input string, as needed to sup
 * options.
 *
 * The options field specifies what operations to *skip*, so kCaseSensitive
 * means to skip case folding and kDiacriticSensitive means to skip diacrit
 * striping. If both flags are specified, the input utf8 StringData is retu
 * directly without any processing or copying.
 *
 * If processing is performed, the returned StringData will be placed in
 * buffer.  buffer's contents (if any) will be replaced. Since we may retur
 * the input unmodified the returned StringData's lifetime is the shorter o
 * the input utf8 and the next modification to buffer. The input utf8 must
 * point into buffer.
 */
static StringData caseFoldAndStripDiacritics(StackBufBuilder* buffer,
                                             StringData utf8,
                                             SubstrMatchOptions options,
                                             CaseFoldMode mode);
```

(Comments re-wrapped.)

§ 2.3. VMware

VMware has an internal string builder implementation that avoids `std::string` due, in part, to `reserve`'s zero-writing behavior. This is similar in spirit to the MongoDB example above.

§ 2.4. Discussion on std-proposals

This topic was discussed in 2013 on std-proposals in a thread titled "Add basic_string::resize_uninitialized (or a similar mechanism)":

https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/XIO4KbBTxl0

§ 2.5. DynamicBuffer

The [N4734] (the Networking TS) has *dynamic buffer* types.

§ 2.6. P1020R0

See also [P1020R0] "Smart pointer creation functions for default initialization".

§ 2.7. Transferring Between `basic_string` and `vector`

This new approach is not limited to avoiding redundant initialization for `basic_string` and `vector`, we also have an efficient way for transferring buffers between the two containers where copying was previously necessary:

```
void Transform() {
    std::string<char> str;
    // Populate and initialize str.

    // SUB-OPTIMAL:
    // * We have an extra allocation/deallocation.
    // * We have to copy the contents of the new buffer, when we could have
    //   reused str's buffer (if the API permitted that expression).
    std::vector<char> vec(str.begin(), str.end());

    // str is not used after this point...
    SomeMethodTakingVector(vec);
}
```

§ 3. Proposal

This paper proposes several options for LEWG's consideration.

§ 3.1. `storage_buffer` as a container

We propose a new container type `std::storage_buffer`

```
namespace std {

template<typename T, typename Allocator = std::allocator<T>>
class storage_buffer {
  public:
    // types
    using value_type             = T;
    using allocator_type         = Allocator;
    using pointer                = T*;
    using const_pointer          = const T*;
    using reference              = T&;
    using const_reference        = const T&;
    using size_type              = *implementation-defined*;
    using iterator               = *implementation-defined*;
    using const_iterator         = *implementation-defined*;
    using reverse_iterator       = *implementation-defined*;
    using const_reverse_iterator = *implementation-defined*;

    // constructors/destructors
    constexpr storage_buffer() noexcept;
    storage_buffer(storage_buffer&& s) noexcept;
    ~storage_buffer();
    allocator_type get_allocator() const noexcept;

    // assignment
    storage_buffer& operator=(storage_buffer&& s) noexcept;
    storage_buffer& operator=(basic_string<T, char_traits<T>, Allocator>&&
    storage_buffer& operator=(vector<T, Allocator>&& v);

    // iterators
    iterator               begin() noexcept;
    const_iterator         begin() const noexcept;
    iterator               end() noexcept;
    const_iterator         end() const noexcept;
    reverse_iterator       rbegin() noexcept;
    const_reverse_iterator rbegin() const noexcept;
    reverse_iterator       rend() noexcept;
    const_reverse_iterator rend() const noexcept;

    const_iterator         cbegin() const noexcept;
    const_iterator         cend() const noexcept;
    const_reverse_iterator crbegin() const noexcept;
    const_reverse_iterator crend() const noexcept;
```

```
    // capacity
    [[nodiscard]] bool empty() const noexcept;
    size_type size() const noexcept;
    size_type max_size() const noexcept;
    size_type capacity() const noexcept;

    void prepare(size_type n);
    void commit(size_type n);

    // element access
    reference       operator[](size_type n);
    const_reference operator[](size_type n) const;
    reference       at(size_type n);
    const_reference at(size_type n) const;
    reference       front();
    const_reference front() const;
    reference       back();
    const_reference back() const;

    // data access
    pointer data() noexcept;
    const_pointer data() const noexcept;

    // modifiers
    void swap(storage_buffer&) noexcept(
      allocator_traits<Allocator>::propagate_on_container_swap::value ||
      allocator_traits<Allocator>::is_always_equal::value);

    // Disable copy
    storage_buffer(const storage_buffer&) = delete;
    storage_buffer& operator=(const storage_buffer&) = delete;
};

}  // namespace std
```

§ 3.1.1. API Surface

One focus of this container is to make it move-only, as proposed during the [post-Rapperswil] review. This reduces the likelihood that we copy types with trap representations (thereby triggering UB).

Uninitialized data is only accessible from the `storage_buffer` type. For an API directly manipulating `basic_string` or `vector`, the invariant that uninitialized data is not available is otherwise weakened.

For purposes of the container API, `size()` corresponds to the *committed* portion of the buffer. This leads to more consistency when working with (and explicitly copying to) other containers via iterators, for example:

```
std::storage_buffer<char> buf;
buf.prepare(100);
*fill in data*
buf.commit(50);

std::string a(buf.begin(), buf.end());
std::string b(std::move(buf));

assert(a == b);
```

Besides the well-known container APIs, `storage_buffer` would have several novel APIs:

```
void prepare(size_type n);
```

- *Effects*: After `prepare()`, `capacity() >= n` and ensures that `[data(), data()+capacity())` is a valid range. Note that `[data()+size(), data()+capacity())` may contain indeterminate values. Reallocation occurs if and only if the current capacity is less than `n`.
- *Complexity*: At most linear time in the size of the sequence.
- *Throws*: `length_error` if `n > max_size()`. `Allocator::allocate` may throw.
- *Remarks*: Reallocation invalidates all references, pointers, and iterators referring to elements of the sequence.

This is similar to `vector::reserve` (see §3.1.2 Bikeshedding), except we need to explicitly guarantee that `[data()+size(),data()+capacity())` is a valid range. For allocation and copy-free transfers into `basic_string`, space for the null terminator should be contemplated in a call to `prepare`.

```
void commit(size_type n);
```

- *Requires*: `n <= capacity() - size()`
- *Effects*: Adds `n` elements to the sequence starting at `data()+size()`.
- *Complexity*: Constant time
- *Remarks*: The application must have been initialized since the proceeding call to `prepare` otherwise the behavior is undefined.

When moving `storage_buffer` into a `basic_string` or `vector`, only the *committed* (`[data(), data()+size())`) contents are preserved.

```
basic_string(storage_buffer&& buf);
```

- *Effects*: Constructs an object of class `basic_string`
- *Ensures*: `data()` points at the first element of an allocated copy of the array whose first element is pointed at by the original value `buf.data()`, `size()` is equal to the original value of `buf.size()`, and `capacity()` is a value at least as large as `size()`. `buf` is left in a valid state with an unspecified value.
- *Remarks*: Reallocation may occur if `buf.size() == buf.capacity()`, leaving no room for the "null terminator" [24.3.2].

```
vector(storage_buffer&& buf);
```

- *Effects*: Constructs an object of class `vector`
- *Ensures*: `data()` points at the first element of an allocated copy of the array whose first element is pointed at by the original value `buf.data()`, `size()` is equal to the original value of `buf.size()`, and `capacity()` is a value at least as large as `size()`. `buf` is left in a valid state with an unspecified value.

§ 3.1.2. Bikeshedding

What names do we want to use for these methods:

Sizing the default initialized region:

- `prepare` as presented (similar to [N4734] the names used for *dynamic buffer*).
- `reserve` for consistency with existing types

Notifying the container that a region has been initialized:

- `commit` as presented (similar to [N4734]).
- `insert_from_capacity`
- `append_from_capacity`
- `resize_uninitialized`, but the elements `[size(), size()+n)` have been initialized
- `extend`

Transferring from `storage_buffer` to `basic_string` and `vector`:

- Use move construction/assignment (as presented)
- Use explicit `attach`/`detach` APIs or something similar, as presented in §3.2.4 Bikeshedding

§ 3.2. `storage_node` as a transfer vehicle

Alternatively, we contemplate a `storage_node` as having a similar role for `basic_string`/`vector` as `node_handle` provides for associative containers.

`storage_node` owns its underlying allocation and is responsible for destroying any objects and deallocating the backing memory via its allocator.

§ 3.2.1. `storage_node` API

```
template<unspecified>
class storage_node {
  public:
    // These type declarations are described in [containers.requirements.ge
    using value_type = see below;
    using allocator_type = see below;

    ~storage_node();
    storage_node(storage_node&&) noexcept;
    storage_node& operator=(storage_node&&);

    allocator_type get_allocator() const;
    explicit operator bool() const noexcept;
    bool empty() const noexcept;
    void swap(storage_node&) noexcept(
        allocator_traits<allocator_type>::propagate_on_container_swap::valu
        allocator_traits<allocator_type>::is_always_equal::value);

    friend void swap(storage_node& x, storage_node& y) noexcept(
        noexcept(x.swap(y))) { x.swap(y); }
};
```

As-presented, this type can only be constructed by the library, rather than the user. To address the redundant initialization problem, we have two routes forward:

- Adopt [P1010R1]'s `uninitialized_data` and `insert_from_capacity` API additions to `vector`.
- Be able to manipulate the constituent components of the transfer node and reassemble it.

```
std::string GeneratePattern(const std::string& pattern, size_t count) {
    const auto step = pattern.size();

    // GOOD:  No initialization
    std::string ret;
    ret.reserve(step * count);

    auto tmp = ret.release();

    char* start = tmp.data();
    auto  size  = tmp.size();
    auto  cap   = tmp.capacity();
    tmp.release();

    for (size_t i = 0; i < count; i++) {
        // GOOD: No bookkeeping
        memcpy(start + i * step, pattern.data(), step);
    }

    return std::string(std::storage_node(
        tmp, size + count * step, cap, alloc));
}
```

(It is important to reiterate that the above implementation is possible because we statically know that the allocator is `std::allocator` and that the `value_type` is `char`. A generic implementation of this pattern would need to be constrained based on allocators, allocator traits, and value types. Future library and language extensions may expand the set of applicable types and may make it easier to constrain generic implementations.)

Allowing users to provide their own `allocator::allocate`'d buffers runs into the "offset problem". Consider an implementation that stores its `size` and `capacity` inline with its data, so `sizeof(vector) == sizeof(void*)`.

```
class container {
  struct Rep {
    size_t size;
    size_t capacity;
  };

  Rep* rep_;
};
```

Going back to our original motivating examples:

```
std::string GeneratePattern(const std::string& pattern, size_t count) {
    const auto step = pattern.size();

    // GOOD:  No initialization
    std::allocator<char> alloc;
    char* tmp = alloc.allocate(step * count + 1);

    for (size_t i = 0; i < count; i++) {
        // GOOD: No bookkeeping
        memcpy(tmp + i * step, pattern.data(), step);
    }

    return std::string(std::storage_node(
        tmp, size * count, size * count + 1, 0 /* offset */, alloc));
}
```

If using a `Rep`-style implementation, the mismatch in offsets requires an O(N) move to shift the contents into place and a possible realloc.

§ 3.2.2. `basic_string` additions

In [**basic.string**] add the declaration for `extract` and `insert`.

In [**string.modifiers**] add new:

`storage_node extract();`

3. *Effects:* Removes the buffer from the string and returns a `storage_node` owning the buffer. The string is left in a valid state with unspecified value.
4. *Complexity:* - Linear in the size of the sequence.

`void insert(storage_node&& buf);`

3. *Requires:* - `buf.empty()` or `get_allocator() == buf.get_allocator()`.
4. *Effects:* If `buf.empty`, has no effect. Otherwise, assigns the buffer owned by `buf`.
5. *Postconditions:* `buf` is empty.
6. *Complexity:* - Linear in the size of the sequence.

§ 3.2.3. `vector` additions

In [**vector.overview**] add the declaration for `extract`, `insert`, and `insert_from_capacity`:

```
namespace std {
  template<class T, class Allocator = allocator<T>>
  class vector {

    ...

    // 26.3.11.4, data access
    T* data() noexcept;
    const T* data() const noexcept;
    T* uninitialized_data() noexcept;

    // 26.3.11.5, modifiers
    template<class... Args> reference emplace_back(Args&&... args);
    void push_back(const T& x);
    void push_back(T&& x);
    void pop_back();

    template<class... Args> iterator emplace(const_iterator position, Args
    iterator insert(const_iterator position, const T& x);
    iterator insert(const_iterator position, T&& x);
    iterator insert(const_iterator position, size_type n, const T& x);
    template<class InputIterator>
      iterator insert(const_iterator position, InputIterator first, Input
    iterator insert(const_iterator position, initializer_list<T> il);
    void insert(storage_node&& buf);
    iterator insert_from_capacity(size_type n);
    iterator erase(const_iterator position);
    iterator erase(const_iterator first, const_iterator last);
    storage_node extract();

    ...
```

In [**vector.data**] add new p3-5:

```
T*         data() noexcept;
const T*   data() const noexcept;
```

1. *Returns:* A pointer such that `[data(), data() + size())` is a valid range. For a non-empty vector, `data() == addressof(front())`.
2. *Complexity:* Constant time.

```
T*         uninitialized_data() noexcept;
```

3. *Returns:* A pointer to uninitialized storage that would hold elements in the range `[size(), capacity())`. [*Note:* This storage may be initialized through a pointer obtained by casting `T*` to `void*` and then to `char*`, `unsigned char*`, or `std::byte*`. ([**basic.life**]p6.4). - *end note*]
4. *Remarks:* This member function does not participate in overload resolution if `allocator_traits<Allocator>::rebind_traits<U>::implicit_construct(U*)` is not well-formed.
5. *Complexity:* Constant time.

In [**vector.modifiers**] add new p3-6:

2. *Complexity:* The complexity is linear in the number of elements inserted plus the distance to the end of the vector.

`iterator insert_from_capacity(size_type n);`

3. *Requires:* - `n <= capacity() - size()`.
4. *Remarks:* - Appends `n` elements by implicitly creating them from capacity. The application must have initialized the storage backing these elements otherwise the behavior is undefined. This member function does not participate in overload resolution if `allocator_traits<Allocator>::rebind_traits<U>::implicit_construct(U*)` is not well-formed.
5. *Returns:* - an iterator to the first element inserted, otherwise `end()`.
6. *Complexity:* - The complexity is linear in the number of elements inserted. [*Note:* For some allocators, including the default allocator, actual complexity is constant time. - *end note*]

`storage_node extract();`

3. *Effects:* Removes the buffer from the container and returns a `storage_node` owning the buffer.
4. *Complexity:* - Constant time.

`void insert(storage_node&& buf);`

3. *Requires:* - `buf.empty()` or `get_allocator() == buf.get_allocator()`.
4. *Effects:* If `buf.empty`, has no effect. Otherwise, assigns the buffer owned by `buf`.
5. *Postconditions:* `buf` is empty.
6. *Complexity:* - Constant time.

```
iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
void pop_back();
```

7. *Effects:* Invalidates iterators and references at or after the point of the erase.
8. ...

§ 3.2.4. Bikeshedding

What should we call the methods for obtaining and using a node?

- `extract` / `insert` for consistency with the APIs of associative containers (added by [P0083R3])
- `detach` / `attach`
- `release` / `reset` for consistency with `unique_ptr`
- `get_storage_node` / `put_storage_node`

3.3. Allocator Support

Allocator aware containers must cooperate with their allocator for object construction and destruction.

3.3.1. Default Initialization

The "container" approach implies adding "`default_construct`" support to the allocator model. Boost allocators, for example, support default initialization of container elements. Or, as discussed below perhaps we can remove the requirement to call `construct`. See below.

3.3.2. Implicit Lifetime Types

Working with the set of implicit lifetime types defined in [P0593R2] requires that the container use a two step interaction with the application. First, the container exposes memory that the application initializes. Second, the application tells the container how many objects were initialized. The container can then tell the allocator about the newly created objects.

References and wording are relative to [N4762].

Starting with [**allocator.requirements**] (Table 33), we add:

- `a.implicit_construct(c)` - This expression informs the allocator *post facto* of an object of implicit lifetime type that has been initialized and implicitly created by the application. This member function, if provided, does not participate in overload resolution unless `C` is an implicit lifetime type. By default it does nothing.

Then in [**allocator.traits**] we add a new *optional* member:

- `implicit_construct(Alloc& a, T* p)` - This member function:
  - Calls `a.implicit_construct(p)` if it is well-formed, otherwise ...

- Does nothing if T is an implicit lifetime type and `a.construct(p)` is *not* well-formed, otherwise ...
- Does not participate in overload resolution.

(The intent is to leave the meaning of allocators which define `construct(T*)` unchanged, but to allow those that don't, including the default allocator, to support `implicit_construct` implicitly.)

3.3.3. Remove `[implicit_]` `construct` and `destroy` ?

As discussed during the [post-Rapperswil] review of [P1072R0], `implicit_construct` balances out the call to `destroy` when working with implicit lifetime types. During this discussion, no motivating examples were suggested for allocators with non-trivial `implicit_construct` / `destroy` methods. This may motivate *not* adding `implicit_construct` for implicit lifetime types and having no corresponding call to `destroy` (see §6 Questions for LEWG), as `allocate` could merely invoke `std::bless` (see [P0593R2]).

4. Design Considerations

The §3.1 storage_buffer as a container API isolates UB (from accessing uninitialized data) into a specific, clearly expressed container type:

- Code reviews can identify `storage_buffer` and carefully vet it. While it is conceivable that a `storage_buffer` could cross an API boundary, such usage could draw further scrutiny. An instance of `vector` being passed across an API boundary would not. Uninitialized bytes (the promise made by `commit`) never escape into the valid range of `basic_string` and `vector`.
- Analysis tools and sanitizers can continue to check accesses beyond `size()` for `vector`. With the §3.2 storage_node as a transfer vehicle API, accesses in the range `[size(), capacity())` are valid. Even if we provide a distinct API for obtaining the not-yet-committed, uninitialized region (`uninitialized_data()`), it will be contiguous with the valid, initialized region (`data() + size() == uninitialized_data()`), so overruns are harder to detect.

The §3.2 storage_node as a transfer vehicle-oriented API avoids introducing yet another, full-fledged container type. `vector` accomplishes the desired goals with fewer additions.

`basic_string` is often implemented with a short string optimization (SSO). When transferring to `vector` (which lacks a short buffer optimization), an allocation may be required. We can expose

knowledge of that in our API (whether the buffer is allocated or not), but the desire to avoid unneeded initialization is much more suited to larger buffers.

§ 5. Alternatives Considered

[P1010R1] and [P1072R0] contemplated providing direct access to uninitialized elements of `vector` and `basic_string` respectively.

- Boost provides a related optimization for vector-like containers, introduced in [SVN r85964] by Ion Gaztañaga.

E.g.: boost/container/vector.hpp:

```
//! <b>Effects</b>: Constructs a vector that will use a copy of allocator a
//!   and inserts n default initialized values.
//!
//! <b>Throws</b>: If allocator_type's allocation
//!   throws or T's default initialization throws.
//!
//! <b>Complexity</b>: Linear to n.
//!
//! <b>Note</b>: Non-standard extension
vector(size_type n, default_init_t);
vector(size_type n, default_init_t, const allocator_type &a)
...
void resize(size_type new_size, default_init_t);
...
```

These optimizations are also supported in Boost Container's `small_vector`, `static_vector`, `deque`, `stable_vector`, and `string`.

- Adding two new methods to `basic_string`: `uninitialized_data` (to bless access beyond `size()`) and `insert_from_capacity` (to update size without clobbering data beyond the existing size). (This was proposed as "Option B" in [P1072R0].)
- Add `basic_string::resize_uninitialized`. This resizes the string, but leaves the elements indexed from `[old_size(), new_size())` default initialized. (This was originally "Option A" in [P1072R0].)

§ 6. Questions for LEWG

- Does LEWG prefer a full-fledged container like `storage_buffer` a limited node transfer type like `storage_node`?
  - If LEWG prefers the transfer type, how do we solve the redundant initialization problem? Additions to `vector`, based on the [P1010R1], or working directly with raw buffers?
  - If LEWG prefers the transfer type, should it be a named type, or should be only a concept?
- What types do we intend to cover (by avoiding initialization costs)?
  - `char`, `unsigned char`, etc.
  - The implicit lifetime types, described by [P0593R2]
  - Trivially relocatable types, described by [P1144]
- Do we need `implicit_construct`, the complement to `destroy`?
- Do we need to add `default_construct` to the allocator model?
- Do we want to be able to provide an externally-allocated buffer and transfer ownership into a `basic_string`/`vector`/`storage_buffer`?
- Do we want to support buffer transfer to user defined types?
- Do we need to support bookkeeping at the head of allocations?

§ 7. History

§ 7.1. R0 → R1

Applied feedback from LEWG [post-Rapperswil] Email Review:

- Shifted to a new vocabulary types: `storage_buffer` / `storage_node`
  - Added presentation of `storage_buffer` as a new container type
  - Added presentation of `storage_node` as a node-like type
- Added discussion of design and implementation considerations.
- Most of [P1010R1] Was merged into this paper.

§ References

§ Informative References

[Abseil]

Abseil. 2018-09-22. URL: https://github.com/abseil/abseil-cpp

[N4734]

Working Draft, C++ Extensions for Networking. 2018-04-04. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4734.pdf

[N4762]

Working Draft, Standard for Programming Language C++. 2018-07-07. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf

[P0083R3]

Splicing Maps and Sets. 2016-06-24. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0083r3.pdf

[P0593R2]

Implicit creation of objects for low-level object manipulation. 2018-02-11. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0593r2.html

[P1010R1]

Container support for implicit lifetime types. . 2018-10-08. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1010r1.html

[P1020R0]

Smart pointer creation with default initialization. 2018-04-08. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1020r0.html

[P1072R0]

Default Initialization for basic_string. 2018-05-04. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1072r0.html

[P1144]

Object relocation in terms of move plus destroy. 2018-07-06. URL: https://quuxplusone.github.io/blog/code/object-relocation-in-terms-of-move-plus-destroy-draft-7.html

[post-Rapperswil]

LEWG Weekly - P1072. 2018-08-26. URL: http://lists.isocpp.org/lib-ext/2018/08/8299.php

[Protobuf]

Protocol Buffers. 2018-09-22. URL: https://github.com/protocolbuffers/protobuf

[Snappy]

Snappy. 2018-09-21. URL: https://github.com/google/snappy